ZAPPLANCE

Freelance Marketplace Platform

Complete Architectural Guide & Learning Roadmap for JavaScript/React Developers

React 18 | TypeScript | Redux Toolkit | Node.js | Express | MongoDB | Socket.io | WebRTC

Designed for: Developers with JS + Basic React Knowledge | Full-Stack Guide

1. Executive Overview & System Architecture

Understanding what Zapplance is and how its frontend, backend, database, and real-time layers work together.

Zapplance is a production-style freelance marketplace platform modeled after services like Upwork and Fiverr. It supports two primary user roles: Clients (who publish job postings, review proposals, negotiate bids, and hire) and Freelancers (who discover projects, submit bids, negotiate counter-offers, and deliver work).

Once a freelancer is hired, the platform unlocks a dedicated collaborative Workspace featuring real-time messaging, file uploads, milestone task tracking, escrow/partial payments simulation, and peer-to-peer live video calls.

Frontend Architecture

- React 18 (Vite build tool) with TypeScript
- React Router v6 for public/protected routes
- Redux Toolkit for global Auth, Projects, Bids
- Tailwind CSS & Shadcn UI (Radix primitives)
- Socket.io Client & WebRTC RTCPeerConnection

Backend & Data Tier

- Node.js & Express.js REST API with MVC pattern
- JWT Authentication & role-based middleware
- MongoDB with Mongoose ODM Schemas
- Socket.IO Server for live chat and signaling
- Multer for multipart disk-based file uploads

Core Mental Model

The frontend is a Single Page Application (SPA) interacting with the backend via two channels: standard REST HTTP requests (via Fetch API / Axios) for data persistence, and persistent WebSockets (Socket.io) + WebRTC for real-time messaging, notifications, and direct video conferencing.

2. The 5 Essential Pillars to Master

Pillar 1: TypeScript (The Codebase Dialect)

If you only know vanilla JavaScript, TypeScript adds static type annotations to catch errors at compile time before running in the browser.

- **Interfaces & Types:** Defines the shape of data models (e.g., interface Project { id: string; title: string; budget: number; status: string; }).
- **Component Props & State:** Ensures components receive expected props (e.g., React.FC<{ project: Project; onClick: () => void }>).
- **Generics <T>:** Allows writing reusable logic that operates over various data types (e.g., Promise<T>, useSelector((state: RootState) => ...)).

Key Files: Frontend/src/lib/api/http.ts | Frontend/src/store/slices/authSlice.ts

Pillar 2: Advanced React & State Management

Beyond basic useState and useEffect, Zapplance uses specialized hooks and centralized global state.

- **React Router v6:** Dynamic URL params (useParams() -> projectId), navigation (useNavigate()), and ProtectedRoute wrappers that check if user is logged in.
- **useRef Hook:** Maintains persistent references to HTML5 video elements (localVideoRef, remoteVideoRef) and active RTCPeerConnection instances without triggering re-renders.
- **Redux Toolkit (RTK):** Stores application-wide state (current logged-in user, active tokens, notifications) so any component can access data without prop-drilling.

```
Key Files: Frontend/src/App.tsx | Frontend/src/store/index.ts |
Frontend/src/pages/Workspace.tsx
```

Pillar 3: Node.js, Express & MongoDB (Backend MVC)

The backend follows the Model-View-Controller (MVC) architecture without server-rendered views.

- **Routes (routes/index.js):** Maps URL endpoints (e.g., POST /api/projects) to middleware and controller functions.
- **Middleware (middleware/auth.js):** Functions that run before controllers. requireAuth checks JWT validity; requireRole("client") blocks freelancers from posting jobs.
- **Controllers (controllers/*):** Executes business logic: validating inputs, running database queries, calculating stats, and returning JSON responses.
- **Mongoose Models (models/*):** Defines database collections and schemas (User, Project, Bid, Payment, Timeline, Message).

```
Key Files: Backend/src/server.js | Backend/src/routes/index.js |
Backend/src/controllers/bidController.js
```

Pillar 4: Real-Time Communication (Socket.IO & WebRTC)

This is the most advanced and impressive part of the application. It powers live chat and video conferencing.

- **Socket.IO (WebSockets):** Maintains a bidirectional TCP connection. Clients emit events (join-room, message), and the server broadcasts them to specific project rooms (io.to(roomId).emit(...)).
- **WebRTC (Video Calling):** Enables browser-to-browser peer-to-peer audio/video streaming without sending heavy video data through the backend server.
- **Signaling Mechanism:** Socket.IO acts as the signaling channel to exchange SDP Offers, SDP Answers, and ICE Candidates between two users so their browsers can establish a direct P2P connection.

```
Key Files: Backend/src/server.js (Lines 86-250) | Frontend/src/pages/Workspace.tsx
```

Pillar 5: Modern UI System (Tailwind CSS & Shadcn UI)

Zapplance features a sleek, responsive design with Bento Grid layouts and micro-animations.

- **Tailwind CSS:** Utility-first styling using pre-built classes for flexbox, grid, spacing, colors, and responsive breakpoints (md:, lg:).
- **Shadcn UI & Radix UI:** Accessible, headless primitives (dialogs, dropdowns, tabs, avatars, tooltips) customized through Tailwind tokens in index.css.

- **Framer Motion:** Declarative animations for smooth page transitions and interactive hover effects.

```
Key Files: Frontend/src/components/ui/* | Frontend/src/components/BentoGrid.tsx |
Frontend/src/index.css
```

3. End-to-End Business Flow & User Journey

Follow this step-by-step lifecycle to understand how data moves through every layer of the app.

- **Phase 1: Registration & Role Selection:** User signs up via /register specifying their role: client or freelancer. Passwords are encrypted with bcryptjs, and a JWT token is returned and stored in localStorage. Redux store saves the user profile.
- **Phase 2: Project Creation & Discovery:** Client accesses /post-project to publish a job with title, description, budget, required skills, and deadlines. Freelancers browse active listings on /projects using either the Bento Grid or List layout with real-time filters.
- **Phase 3: Bidding, Proposals & Counter-Offers:** Freelancer opens /projects/:id and submits a proposal specifying their bid amount, delivery time, and cover letter. Clients review bids, can reject, send counter-offers, or accept the proposal.
- **Phase 4: Hiring & Workspace Activation:** When the Client clicks "Hire", the project status updates to "in_progress". The collaborative Workspace (/workspace/:projectId) is unlocked exclusively for the project owner and hired freelancer.
- **Phase 5: Collaborative Workspace & Live Calling:** Inside the workspace, both parties communicate via real-time Socket.io chat, upload attachments (handled via Multer), and launch 1-on-1 WebRTC video calls with camera/mic controls.
- **Phase 6: Milestone Tasks & Payment Release:** The project timeline breaks work into milestone tasks. As tasks are completed, partial milestone payments are released (simulated escrow), and clients submit final ratings and reviews.

4. Guided File-by-File Reading Order

Read the codebase in this precise sequence to build your mental model effortlessly.

1. Frontend Routing & Access Control

`Frontend/src/App.tsx & Frontend/src/components/ProtectedRoute.tsx`

See how client-side routing works. Notice how ProtectedRoute checks the Redux auth state and redirects unauthenticated users to /login.

2. Global State & API Communications

`Frontend/src/store/slices/authSlice.ts & Frontend/src/lib/api/http.ts`

Understand how login tokens are stored in localStorage and how the custom http wrapper automatically injects the Authorization: Bearer <token> header.

3. Backend Server & API Routes

`Backend/src/server.js & Backend/src/routes/index.js`

Inspect the Express setup, MongoDB connection, CORS rules, and the master route definitions connecting HTTP URLs to controllers.

4. Database Models

`Backend/src/models/User.js, Project.js, Bid.js, Timeline.js, Payment.js`

Examine the Mongoose schemas to understand how data is structured and how documents reference each other via ObjectIds.

5. Core Feature Controllers

`Backend/src/controllers/authController.js, projectController.js, bidController.js`

See the exact logic for creating accounts, filtering projects, creating bids, submitting counter-offers, and executing the hire action.

6. The Real-Time Workspace

`Frontend/src/pages/Workspace.tsx & Backend/src/server.js (Socket logic)`

Study how socketRef and RTCPeerConnection are initialized, how messages are broadcasted, and how WebRTC video streams are linked to <video> tags.

5. Actionable 2-Week Learning Plan

Follow this day-by-day plan to progress from JavaScript/basic React to understanding the full Zapplance stack.

- **Days 1 - 3: TypeScript & React Router v6:** Learn TypeScript interfaces, type aliases, and generic types. Practice typing React props and hooks. Study dynamic routes (useParams) and route guards.
- **Days 4 - 5: Redux Toolkit (RTK):** Learn createSlice, reducers, actions, useSelector, and useDispatch. Trace how user login updates state across all navbar and dashboard components.
- **Days 6 - 8: Node.js, Express & MongoDB:** Understand Express routing, middleware, JWT verification, and Mongoose CRUD queries with .populate(). Test endpoints using Postman or Thunder Client.
- **Days 9 - 11: WebSockets with Socket.IO:** Learn the difference between HTTP request/response and persistent WebSockets. Practice emitting and listening to custom events and room-based broadcasts.
- **Days 12 - 14: WebRTC & Full Project Review:** Understand the WebRTC handshake: getUserMedia, Offer/Answer SDP exchange, ICE candidates, and attaching MediaStreams to video elements. Run the project locally and test the entire hiring cycle.

Summary Recommendation

Start by cloning the environment, running "npm run dev" in both Frontend and Backend, creating two accounts (one Client, one Freelancer in different browser windows or Incognito), and walking through the post -> bid -> hire -> workspace cycle while watching the terminal logs.